

Week 6 Assignment

Name : Labhade Siddharth Popat

Student ID: CT_CSI_DE_1176

College Name: Sanjivani College Of Engineering Kopergaon

Spark Assignment

Objective:

Understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, and optimized file formats. Steps: Understand Spark architecture (Driver, Cluster Manager, Executors) and execution modes. Learn Lazy Evaluation and how it optimizes execution using DAG (Lineage Graph). Read data from files (CSV, Parquet) with proper schema handling. Perform filtering and selection of required columns. Modify DataFrames (rename columns, cast data types, add new columns). Apply transformations and actions appropriately. Understand wide transformations and performance concepts (Shuffle, Predicate Pushdown). Work with different file formats (CSV vs Parquet) and their impact on performance. Handle null values and filter datasets efficiently. Build data pipelines (read → transform → filter → write). Save processed data into required formats (CSV/Parquet). Follow best practices for large datasets (avoid collect(), use show()). Output: Spark code (PySpark/Scala) + execution results + brief insights on performance and architecture.

Resources :

https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQDLvM3UIKsvSLKuDW6nbDK9AaeiXu7C2zJOow_bWTU6WNo?e=nwIp8h

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Apache Spark follows a **master-worker architecture** to process large datasets in a distributed environment. A Spark application mainly consists of three important components: the **Driver**, the **Cluster Manager**, and the **Executors**. Each has a specific role in the execution of the application.

1. Driver

The **Driver** is the central component of every Spark application. It acts as the **brain** of the application because it controls and coordinates the entire execution process.

When a Spark program starts, the Driver creates the **SparkSession** (or SparkContext in older versions), which serves as the entry point for interacting with Spark.

Responsibilities of the Driver

- Creates the SparkSession/SparkContext.

- Reads and interprets the Spark application code.
- Builds the execution plan (DAG – Directed Acyclic Graph).
- Divides the job into smaller tasks.
- Sends these tasks to Executors.
- Collects results returned by Executors.
- Monitors the progress of the application.

Example

Suppose you write:

```
df = spark.read.csv("sales.csv", header=True)
df.filter(df.amount > 1000).show()
```

The Driver:

1. Reads your code.
2. Creates a logical execution plan.
3. Converts it into physical execution stages.
4. Sends tasks to Executors.
5. Receives the filtered results.
6. Displays the output using `.show()`.

Key Points

- Only **one Driver** exists for each Spark application.
- It coordinates the entire application.
- If the Driver fails, the Spark application stops.

2. Cluster Manager

The **Cluster Manager** is responsible for **resource management**. It allocates CPU cores and memory to Spark applications and launches Executors on worker nodes.

The Driver communicates with the Cluster Manager whenever it needs resources.

Responsibilities of the Cluster Manager

- Allocates cluster resources.
- Starts Executor processes on worker machines.
- Manages CPU and memory allocation.
- Monitors available resources.
- Supports multiple Spark applications running simultaneously.

Types of Cluster Managers

1. **Standalone Cluster Manager**
 - Built into Spark.
 - Easy to configure.
 - Suitable for small clusters.

2. **YARN (Yet Another Resource Negotiator)**
 - Used with Hadoop.
 - Shares resources among multiple applications.
 - Common in enterprise environments.
3. **Apache Mesos**
 - General-purpose cluster manager.
 - Can manage Spark and other distributed frameworks.
4. **Kubernetes**
 - Container orchestration platform.
 - Widely used for cloud deployments.
 - Provides scalability and automated resource management.

Example

If your Spark application requires:

- 4 Executors
- 4 CPU cores each
- 8 GB RAM each

The Driver requests these resources, and the Cluster Manager allocates them on available worker nodes.

3. Executor

An **Executor** is a process that runs on a **Worker Node**. Executors perform the actual computation on the data.

Each Executor receives tasks from the Driver and executes them independently.

Responsibilities of Executors

- Execute tasks assigned by the Driver.
- Read and process partitions of the dataset.
- Store intermediate results in memory.
- Cache DataFrames or RDDs when requested.
- Return computation results to the Driver.

Example

Suppose a dataset contains **10 million records** divided into **10 partitions**.

Spark may create **5 Executors**.

Each Executor processes two partitions in parallel, significantly reducing execution time compared to processing all records sequentially.

Key Points

- Multiple Executors can run in one Spark application.

- Executors run on worker machines.
- If an Executor fails, Spark recomputes only the lost partitions using the lineage graph.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Lazy Evaluation is a feature of Apache Spark where **transformations are not executed immediately**. Instead, Spark records all transformations and waits until an **Action** (such as `show()`, `collect()`, or `count()`) is called. During this time, Spark creates a **Directed Acyclic Graph (DAG)** of all operations and optimizes it before execution.

Example

```
df = spark.read.csv("sales.csv", header=True)

filtered_df = df.filter(df.amount > 1000)

selected_df = filtered_df.select("customer_name", "amount")

selected_df.show()
```

In the above code:

- `read()`, `filter()`, and `select()` are **transformations**, so Spark only records these operations.
- The execution starts only when `show()` is called because it is an **action**.

How Lazy Evaluation Improves Performance

- **Optimizes the execution plan:** Spark combines multiple transformations into a single optimized workflow using the DAG.
- **Reduces disk I/O:** Data is read only when required, avoiding multiple scans of the dataset.
- **Avoids unnecessary computations:** Spark skips operations that are not needed for the final result.
- **Improves memory usage:** Intermediate results are not stored unless required.
- **Enables parallel processing:** After optimization, Spark distributes tasks efficiently across multiple executors.

Code :

Transformation 1

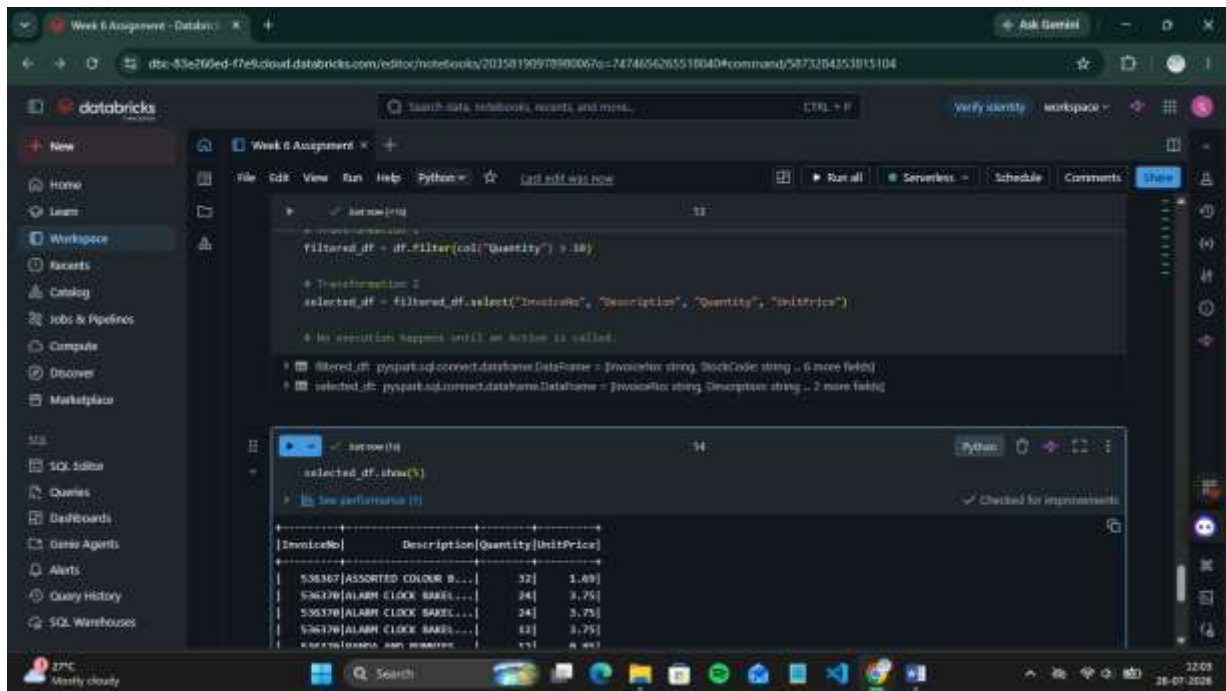
```
filtered_df = df.filter(col("Quantity") > 10)
```

Transformation 2

```
selected_df = filtered_df.select("InvoiceNo", "Description", "Quantity", "UnitPrice")
```

No execution happens until an Action is called.

Output:



Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

To read a CSV file in Apache Spark while treating the first row as column headers and automatically detecting the data types of each column, use the following command:

Code:

```

df = spark.read \

    .option("header", "true") \

    .option("inferSchema", "true") \

    .csv("/Volumes/workspace/default/ecommerce_volume/data.csv")

display(df)

```

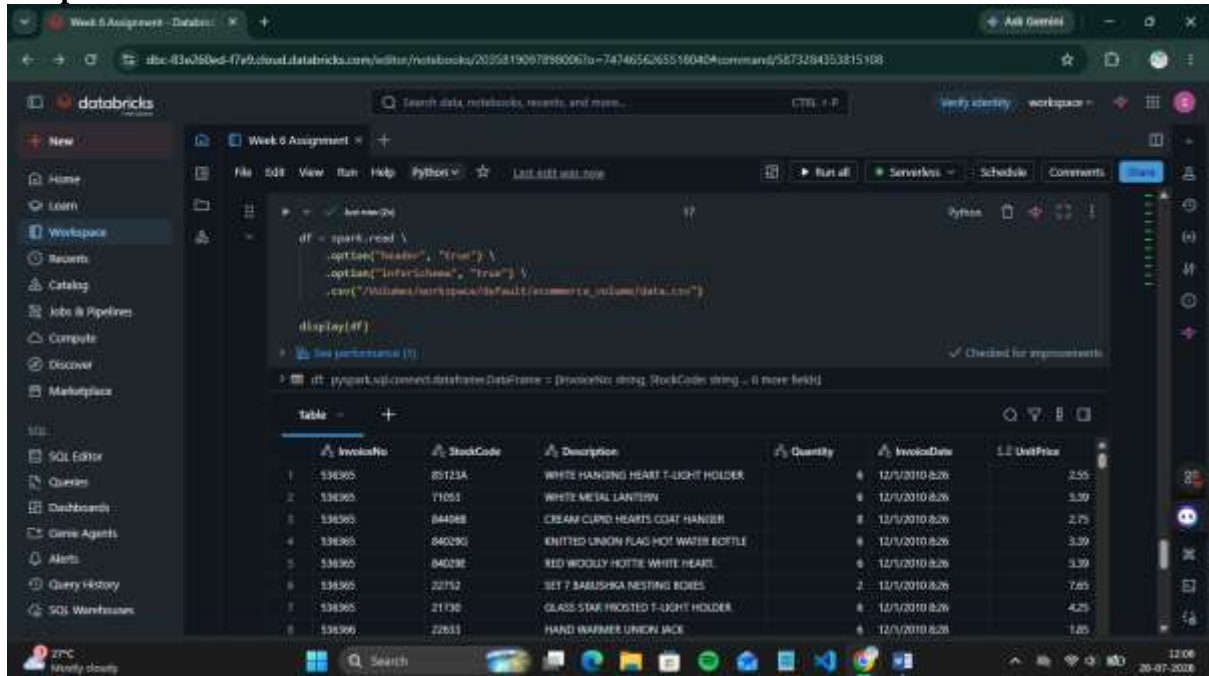
Explanation

- **spark.read:** Used to read data from external sources.
- **.option("header", "true"):** Treats the first row of the CSV file as column names instead of data.
- **.option("inferSchema", "true"):** Automatically detects the data type of each column (e.g., Integer, Double, String) instead of reading all columns as strings.
- **.csv("data/source.csv"):** Specifies the path of the CSV file to be loaded.

Benefits

- Makes the dataset easier to work with by assigning meaningful column names.
- Eliminates the need to manually define the schema.
- Ensures numerical and date columns are read with the correct data types, making data analysis more efficient.

Output:



Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer

CSV and Parquet are two common file formats used for storing data, but they differ in how the data is organized.

CSV (Row-Based Storage)

- CSV stores data **row by row**.
- It is a plain text format that is easy to read and share.
- All columns are stored together for each row.
- It does not store schema information, so Spark often needs to infer or define the schema.

Parquet (Columnar Storage)

- Parquet stores data **column by column**.
- It is a binary file format optimized for big data processing.
- It stores schema information and supports compression.
- Spark can read only the required columns instead of the entire dataset.

Why Does It Matter for Performance?

- **Parquet provides faster query performance** because it reads only the required columns.
- It uses **compression**, reducing storage space and disk I/O.
- It consumes **less memory** and improves processing speed, especially for large datasets.
- **CSV is slower** for analytical queries because Spark must read the entire row, even if only a few columns are needed.

Comparison Table

CSV	Parquet
Row-based storage	Column-based storage
Plain text format	Binary format
Larger file size	Smaller due to compression
Slower read performance	Faster read performance
No schema stored	Stores schema information
Best for data exchange	Best for big data analytics

Conclusion

CSV is simple and widely used for data exchange, while Parquet is more efficient for big data processing. Due to its **columnar storage, compression, and faster query performance**, Parquet is the preferred format in Apache Spark for analytical workloads.

Code:

```
df.write \

    .mode("overwrite") \

    .parquet("/Volumes/workspace/default/ecommerce_volume/parquet_data")

parquet_df = spark.read.parquet(

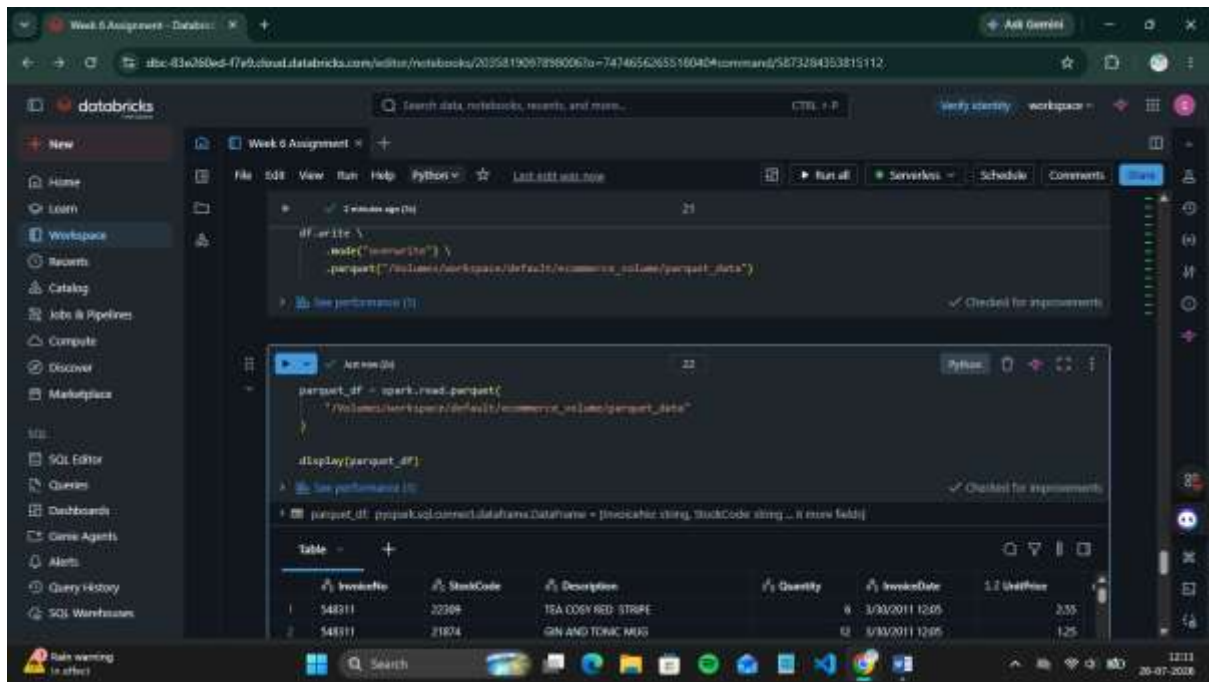
    "/Volumes/workspace/default/ecommerce_volume/parquet_data"

)

parquet_df.printSchema()

display(parquet_df)
```

Output:



Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Answer

The following Spark command selects the product_id and price columns from the DataFrame where the category is "**Electronics**".

from pyspark.sql.functions import col

```
df.select("product_id", "price") \
  .filter(col("category") == "Electronics")
```

Explanation

- **select("product_id", "price"):** Selects only the product_id and price columns from the DataFrame.
- **filter(col("category") == "Electronics"):** Filters the rows where the category column has the value "**Electronics**".
- **col():** Refers to a DataFrame column and is imported from pyspark.sql.functions.

Output

The result will contain only the **product_id** and **price** of products that belong to the **Electronics** category.

Example Output:

product_id price

P101 25000

P205 45000

P310 15000

This query is useful when you need to retrieve specific columns from a DataFrame based on a given condition.

Code:

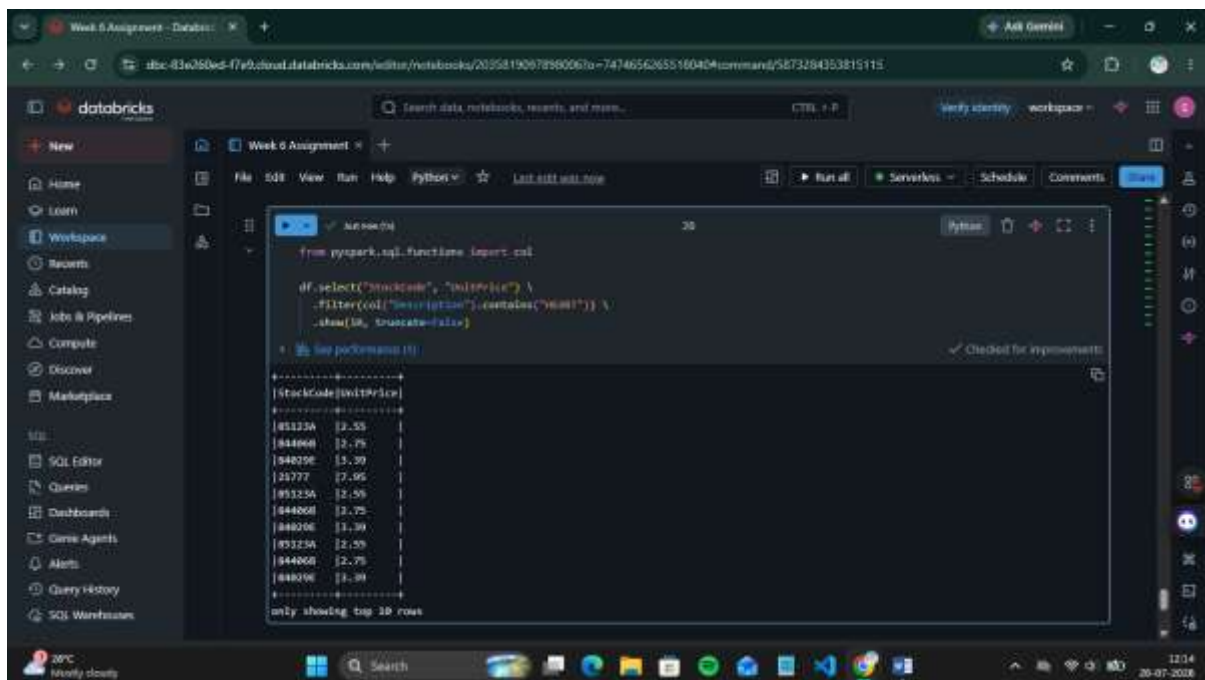
```
from pyspark.sql.functions import col

df.select("StockCode", "UnitPrice") \

.filter(col("Description").contains("HEART")) \

.show(10, truncate=False)
```

Output:



Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Answer

The following Spark code renames the column `old_name` to `new_name` and converts the price column from **String** to **Double**.

```
from pyspark.sql.functions import col

df = df.withColumnRenamed("old_name", "new_name") \
    .withColumn("price", col("price").cast("double"))
```

Explanation

- **withColumnRenamed("old_name", "new_name")**: Renames the existing column `old_name` to `new_name`.
- **withColumn("price", ...)**: Updates the price column with a new data type.
- **cast("double")**: Converts the price column from **String** to **Double**, allowing numerical calculations such as addition, multiplication, and aggregation.

Why is Casting Required?

If the price column is stored as a String, Spark treats it as text, making mathematical operations impossible. By casting it to **Double**, the column can be used for calculations and analysis.

Output

The revised DataFrame will have:

- `old_name` renamed to `new_name`.
- price stored as a **Double** instead of a **String**, making it suitable for numerical operations.

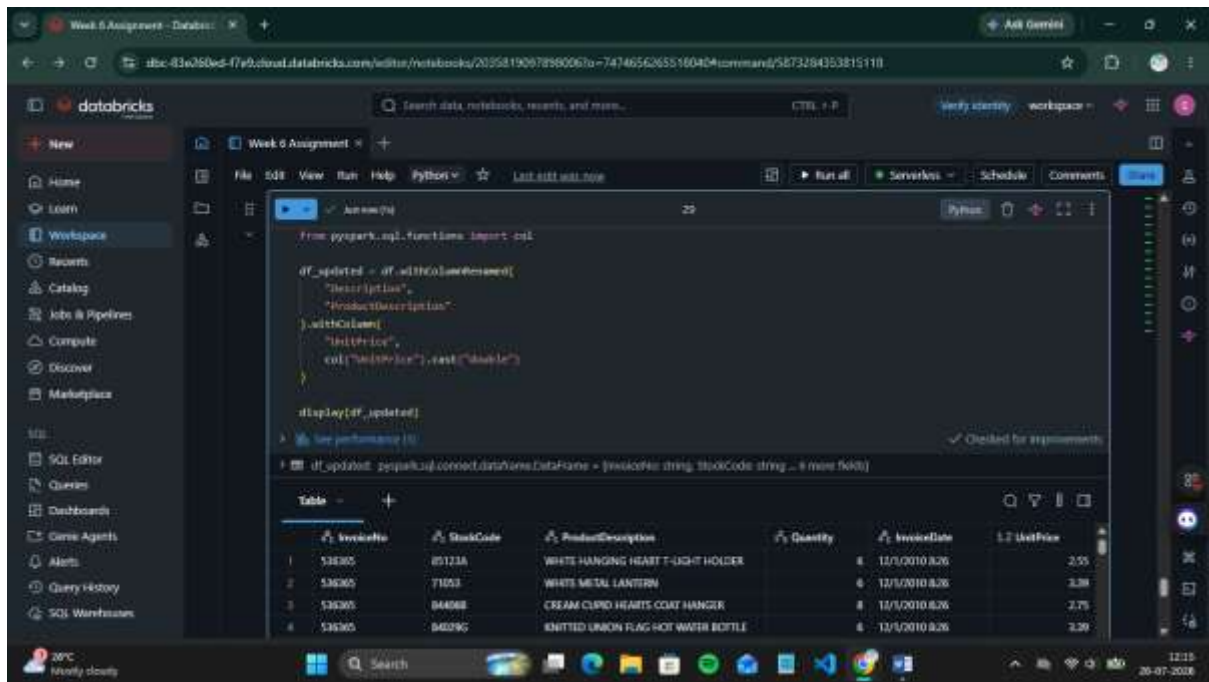
Code:

```
from pyspark.sql.functions import col

df_updated = df.withColumnRenamed(
    "Description",
    "ProductDescription"
).withColumn(
    "UnitPrice",
    col("UnitPrice").cast("double")
)

display(df_updated)
```

Output:



Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer

Apache Spark provides **fault tolerance** using a **Lineage Graph**, also known as a **Directed Acyclic Graph (DAG)**. Instead of storing multiple copies of data, Spark keeps track of all the transformations applied to a dataset.

If a **worker node (Executor)** fails and some data is lost, Spark uses the lineage graph to **recompute only the lost partitions** from the original data, rather than reprocessing the entire dataset.

How It Works

1. Spark records every transformation (such as filter(), select(), and groupBy()) in a DAG.
2. When an action is called, Spark executes the tasks based on this DAG.
3. If an Executor fails, Spark identifies the missing partitions.
4. Using the lineage information, Spark recomputes only those lost partitions and continues execution.

Benefits

- Provides automatic fault tolerance.
- Recomputes only the lost data instead of the entire dataset.
- Reduces recovery time and improves efficiency.
- Ensures reliable processing of large datasets.

Conclusion

The Lineage Graph enables Spark to recover from worker node failures by recreating only the missing data partitions. This makes Spark highly reliable and efficient without requiring data replication.

Code:

```
from pyspark.sql.functions import col

# Transformation 1

filtered_df = df.filter(col("Quantity") > 5)

# Transformation 2

selected_df = filtered_df.select(

    "InvoiceNo",

    "StockCode",

    "Quantity",

    "UnitPrice"

)

# Transformation 3

final_df = selected_df.withColumn(

    "TotalAmount",

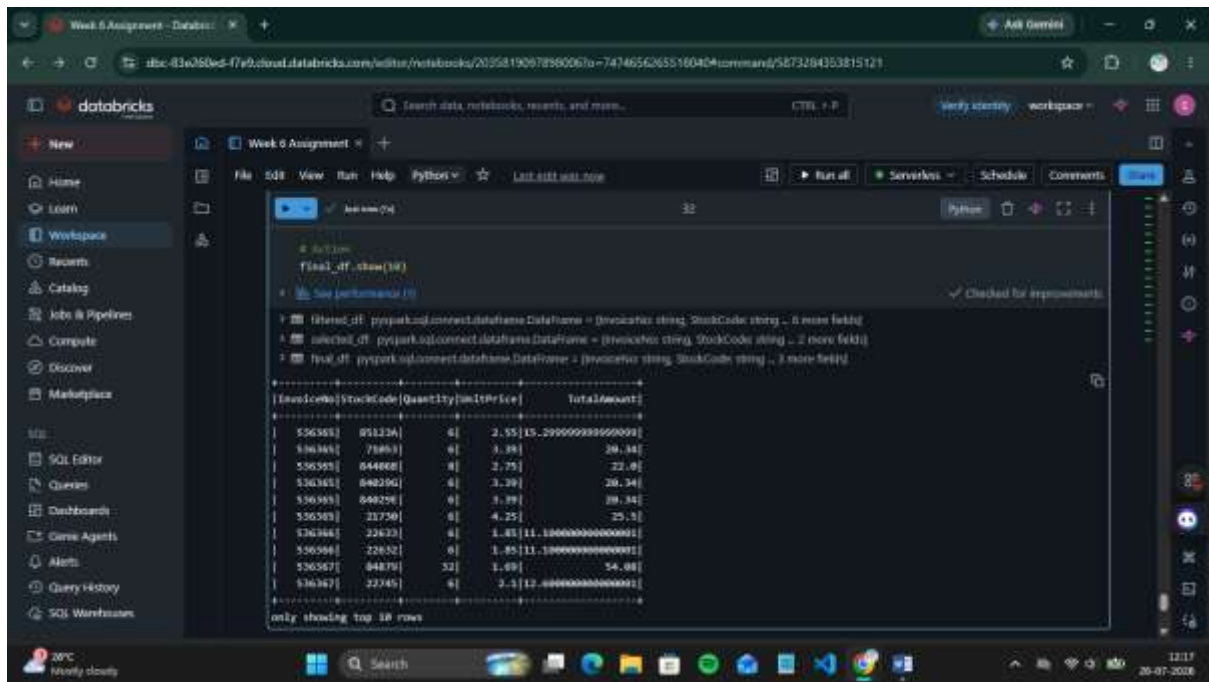
    col("Quantity") * col("UnitPrice")

)

# Action

final_df.show(10)
```

Output:



Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Answer

The following Spark query filters the DataFrame to display only those rows where the **status** is "Completed" and the **amount is greater than 1000**.

```
from pyspark.sql.functions import col
```

```
df_orders.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
)
```

Explanation

- **filter():** Used to filter rows based on a specified condition.
- **col("status") == "Completed":** Selects rows where the status column has the value "Completed".
- **col("amount") > 1000:** Selects rows where the amount is greater than 1000.
- **& (AND operator):** Ensures that **both conditions** must be true for a row to be included in the result.

Example

Input Data:

Order ID	Status	Amount
----------	--------	--------

101	Completed	1500
-----	-----------	------

Order ID	Status	Amount
----------	--------	--------

102	Pending	2000
-----	---------	------

103	Completed	800
-----	-----------	-----

104	Completed	2500
-----	-----------	------

Output:

Order ID	Status	Amount
----------	--------	--------

101	Completed	1500
-----	-----------	------

104	Completed	2500
-----	-----------	------

Conclusion

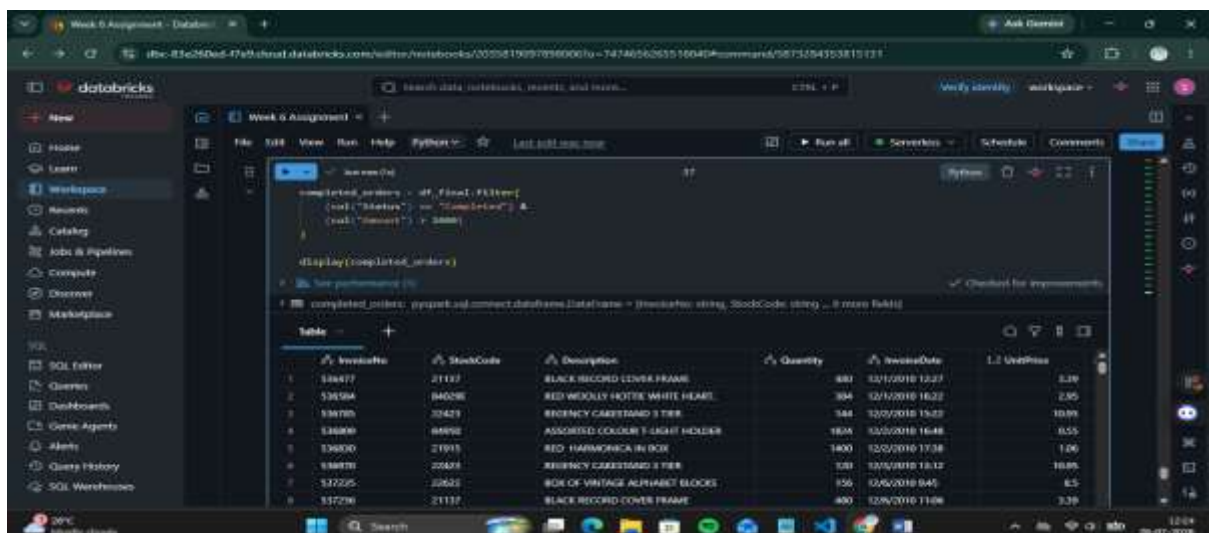
This query filters the DataFrame based on **multiple conditions** using the **AND (&) operator**, returning only the orders that are **completed** and have an **amount greater than 1000**.

Code:

```
completed_orders = df_final.filter(
    (col("Status") == "Completed") &
    (col("Amount") > 1000)
)
```

```
display(completed_orders)
```

Output:



The screenshot shows the Databricks interface with a Python script in the center pane. The script filters a DataFrame for completed orders with an amount greater than 1000. The output is displayed as a table in the bottom pane.

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
1	586477	2112	BLACK RECORD COVER FRAME	880	12/1/2010 12:27	3.39
2	536594	84026	RED WOOLLY HOTTIE WHITE HEART	304	12/1/2010 16:22	2.95
3	536785	20429	BIGNEY CACKETARD 3 TIER	144	12/5/2010 15:22	10.99
4	536809	84892	ASSORTED COLOUR T-LIGHT HOLDER	1804	12/5/2010 16:48	0.55
5	536800	21915	RED HARMONICA IN BOX	1400	12/5/2010 17:38	1.06
6	536870	20429	BIGNEY CACKETARD 3 TIER	1581	12/5/2010 18:12	10.99
7	537225	22622	BOX OF VINTAGE ALPHABET BLOCKS	156	12/6/2010 9:45	4.5
8	537236	21112	BLACK RECORD COVER FRAME	480	12/6/2010 11:08	3.39

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer

Predicate Pushdown is an optimization technique used by Apache Spark when reading **Parquet** files. It allows Spark to apply filter conditions **while reading the data**, instead of loading the entire dataset into memory first.

This means Spark reads **only the rows that satisfy the filter condition** and skips the unnecessary data.

How It Works

For example, if you write:

```
df.filter(col("age") > 30)
```

When the data is stored in **Parquet** format, Spark pushes this filter down to the Parquet reader. As a result, only the relevant data is read from the disk, while the remaining data is ignored.

Benefits

- Reduces the amount of data loaded into memory.
- Minimizes disk I/O by reading only the required data.
- Improves query performance and execution speed.
- Saves memory and CPU resources, especially when working with large datasets.

Conclusion

Predicate Pushdown makes Spark more efficient by filtering data at the storage level itself. This reduces memory usage and speeds up data processing, making Parquet an excellent choice for big data analytics.

Code:

```
parquet_df = spark.read.parquet(

    "/Volumes/workspace/default/ecommerce_volume/parquet_data"

)

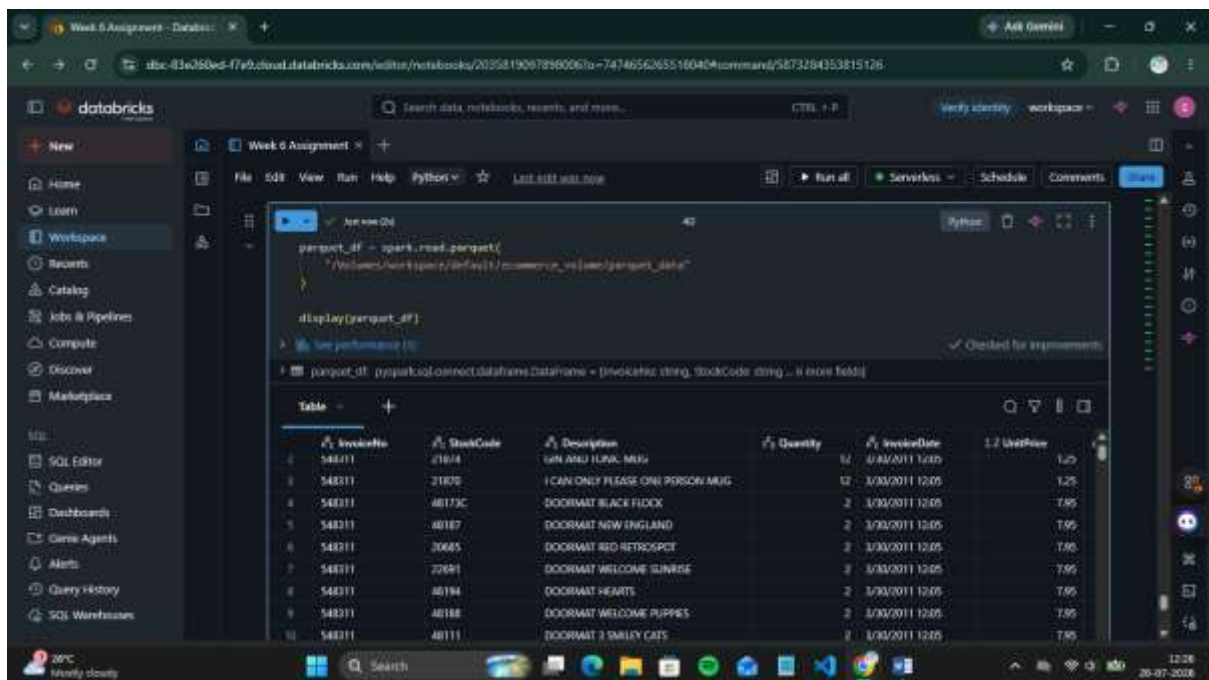
display(parquet_df)


filtered_df = parquet_df.filter(col("Country") == "United Kingdom")

display(filtered_df)
```

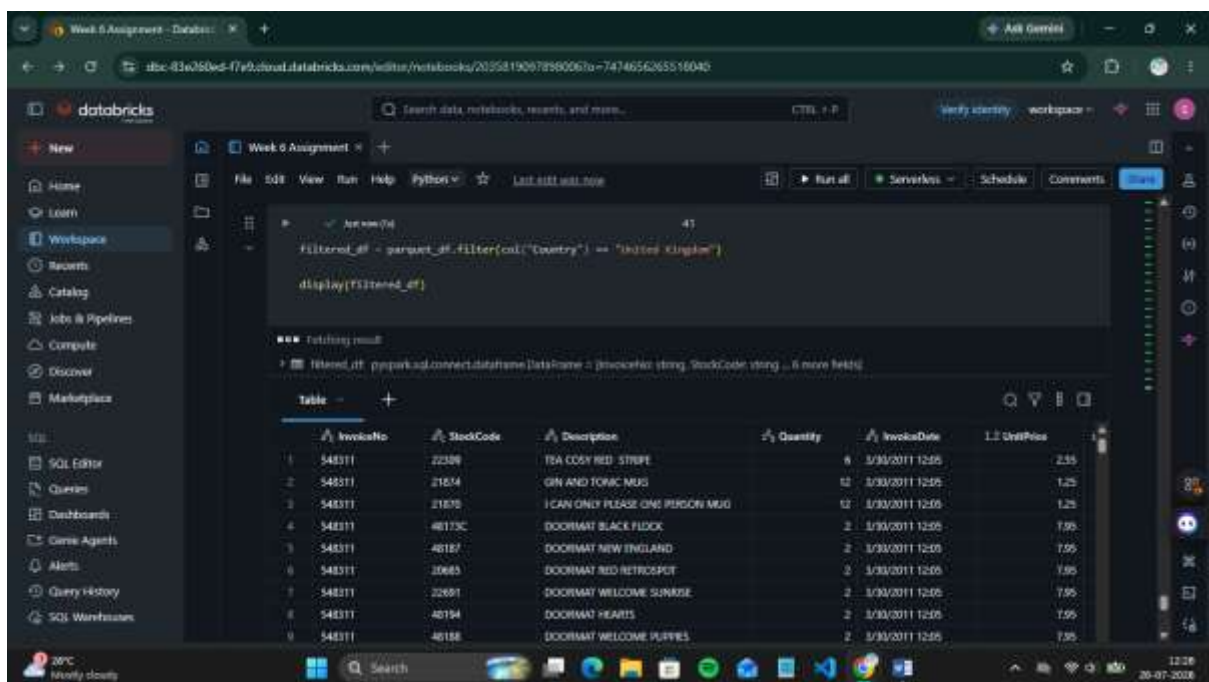
filtered_df.explain(True)

Output:



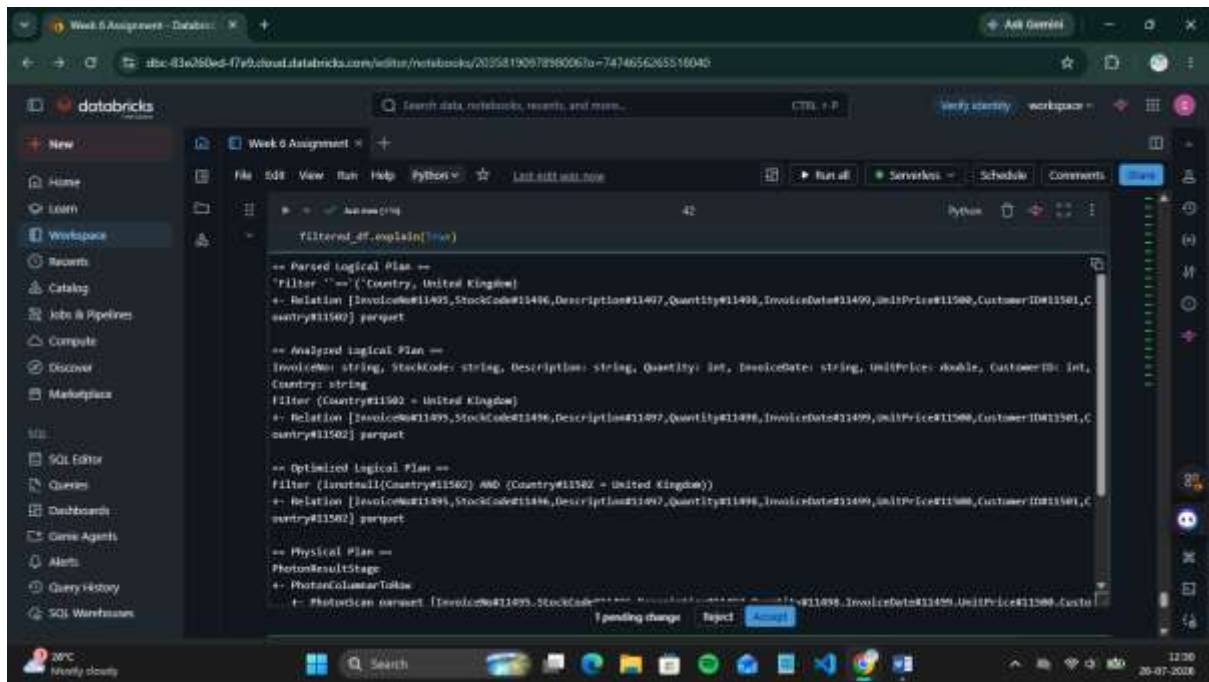
The screenshot shows a Databricks workspace with a Python notebook. The code in the cell reads a parquet file from the workspace and displays the first 10 rows of the resulting DataFrame. The output is a table with 7 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
548311	21814	GIN AND TONIC MUG	12	3/30/2011 12:05	1.25
548311	21870	I CAN ONLY PLEASE ONE PERSON MUG	12	3/30/2011 12:05	1.25
548311	48173C	DOORMAT BLACK FLOXX	2	3/30/2011 12:05	7.95
548311	48187	DOORMAT NEW ENGLAND	2	3/30/2011 12:05	7.95
548311	20685	DOORMAT RED RETROSPCT	2	3/30/2011 12:05	7.95
548311	22691	DOORMAT WELCOME SUNRISE	2	3/30/2011 12:05	7.95
548311	48194	DOORMAT HEARTS	2	3/30/2011 12:05	7.95
548311	48188	DOORMAT WELCOME PUPPES	2	3/30/2011 12:05	7.95
548311	48111	DOORMAT 2 DAILEY CATS	2	3/30/2011 12:05	7.95



The screenshot shows a Databricks workspace with a Python notebook. The code in the cell filters the DataFrame by 'Country' to only include 'United Kingdom' and displays the first 10 rows of the resulting DataFrame. The output is a table with 7 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
548311	22398	TEA COSY RED STRIPE	6	3/30/2011 12:05	2.55
548311	21814	GIN AND TONIC MUG	12	3/30/2011 12:05	1.25
548311	21870	I CAN ONLY PLEASE ONE PERSON MUG	12	3/30/2011 12:05	1.25
548311	48173C	DOORMAT BLACK FLOXX	2	3/30/2011 12:05	7.95
548311	48187	DOORMAT NEW ENGLAND	2	3/30/2011 12:05	7.95
548311	20685	DOORMAT RED RETROSPCT	2	3/30/2011 12:05	7.95
548311	22691	DOORMAT WELCOME SUNRISE	2	3/30/2011 12:05	7.95
548311	48194	DOORMAT HEARTS	2	3/30/2011 12:05	7.95
548311	48188	DOORMAT WELCOME PUPPES	2	3/30/2011 12:05	7.95



Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Answer

The following Spark code creates a new column named `final_price` by multiplying the `base_price` column by **1.18**, which adds an **18% tax**.

from pyspark.sql.functions import col

df = df.withColumn("final_price", col("base_price") * 1.18)

Explanation

- **withColumn():** Adds a new column or updates an existing column in the DataFrame.
- **col("base_price"):** Refers to the `base_price` column.
- *** 1.18:** Multiplies the base price by **1.18**, adding an 18% tax.
- The result is stored in the new **final_price** column.

Example

Input Data:

base_price

1000

2500

5000

Output Data:

base_price	final_price
------------	-------------

1000	1180
------	------

2500	2950
------	------

5000	5900
------	------

Conclusion

This code uses the **withColumn()** function to create a new column by performing a mathematical operation on an existing column. It is commonly used in Spark for calculations such as tax, discounts, and total price.

Code:

```
from pyspark.sql.functions import col
```

```
df_final = df.withColumnn(
```

```
    "FinalPrice",
```

```
    col("UnitPrice") * 1.18
```

```
)
```

```
display(df_final)
```

```
df_final.select(
```

```
    "StockCode",
```

```
    "Description",
```

```
    "UnitPrice",
```

```
    "FinalPrice"
```

```
).show(10, truncate=False)
```

Output:

The screenshot shows a Databricks workspace with a notebook titled "Week 6 Assignment". The notebook contains a Python code block that performs a transformation on a DataFrame. The code is as follows:

```
from pyspark.sql.functions import col

df_final = df.select(
    "FinalPrice",
    col("UnitPrice") * 1.18
)

display(df_final)
```

Below the code, a table view displays the results of the transformation. The table has the following columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice. The data is as follows:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
1	851234	WHITE HANGING HEART T-LIGHT HOLDER	4	12/1/2010 8:26	2.55
2	71603	WHITE METAL LANTERN	4	12/1/2010 8:26	3.39
3	844068	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75
4	848296	KNITTED UNION FLAG HOT WATER BOTTLE	8	12/1/2010 8:26	3.39
5	848296	RED WOOLLY HOTTIE WHITE HEART.	8	12/1/2010 8:26	3.39
6	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65

The screenshot shows a Databricks workspace with a notebook titled "Week 6 Assignment". The notebook contains a Python code block that performs an action on a DataFrame. The code is as follows:

```
df_final.select(
    "StockCode",
    "Description",
    "UnitPrice",
    "FinalPrice"
).show(10, truncate=False)
```

Below the code, a table view displays the results of the action. The table has the following columns: StockCode, Description, UnitPrice, and FinalPrice. The data is as follows:

StockCode	Description	UnitPrice	FinalPrice
851234	WHITE HANGING HEART T-LIGHT HOLDER	2.55	3.0099999999999995
71603	WHITE METAL LANTERN	3.39	4.000333333333333
844068	CREAM CUPID HEARTS COAT HANGER	2.75	3.244333333333333
848296	KNITTED UNION FLAG HOT WATER BOTTLE	3.39	4.000333333333333
848296	RED WOOLLY HOTTIE WHITE HEART.	3.39	4.000333333333333
22752	SET 7 BABUSHKA NESTING BOXES	7.65	9.027
25738	GLASS STAR FROSTED T-LIGHT HOLDER	4.25	5.015
22853	HAND WARMER UNION JACK	1.95	2.303
22632	HAND WARMER RED POLKA DOT	1.95	2.303
84879	ASSORTED COLOUR BIRD ORNAMENT	1.69	1.994333333333333

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Answer

In Apache Spark, operations are classified into **Transformations** and **Actions**.

Transformations

Transformations are operations that create a **new DataFrame or RDD** from an existing one. They are **lazy**, meaning Spark does not execute them immediately. Instead, Spark records these operations and executes them only when an action is called.

Examples:

- filter()
- select()

Actions

Actions are operations that **trigger the execution** of all the previously defined transformations. They either return a result to the Driver or write data to storage.

Examples:

- show()
- count()

Difference Between Transformations and Actions

Transformations	Actions
Create a new DataFrame/RDD	Trigger execution of transformations
Executed lazily	Executed immediately
Return another DataFrame/RDD	Return a value or display/save data
Used to modify or process data	Used to produce the final result

Conclusion

Transformations define **what operations** need to be performed, while Actions **execute** those operations and produce the final output. Spark's lazy execution of transformations helps optimize performance before an action is performed.

Code :

```
from pyspark.sql.functions import col

# Transformation 1

filtered_df = df.filter(col("Quantity") > 10)


# Transformation 2
```

```
selected_df = filtered_df.select(

    "InvoiceNo",

    "StockCode",

    "Quantity",

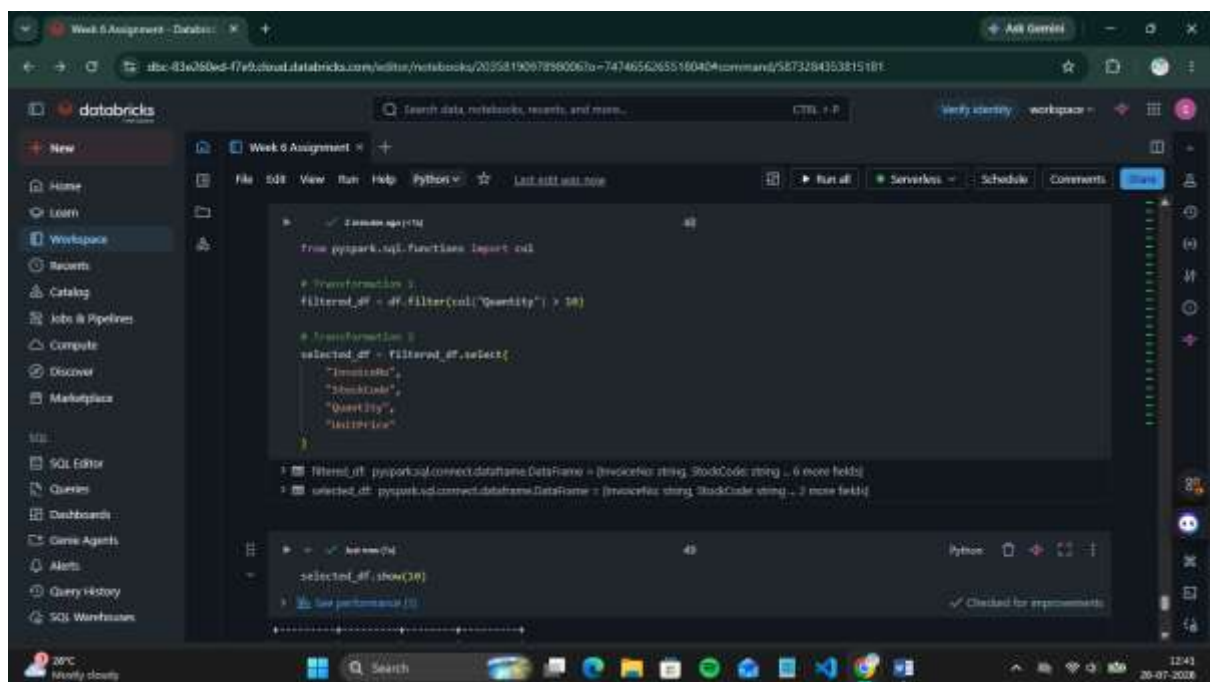
    "UnitPrice"

)
```

```
selected_df.show(10)
```

```
selected_df.count()
```

Output:



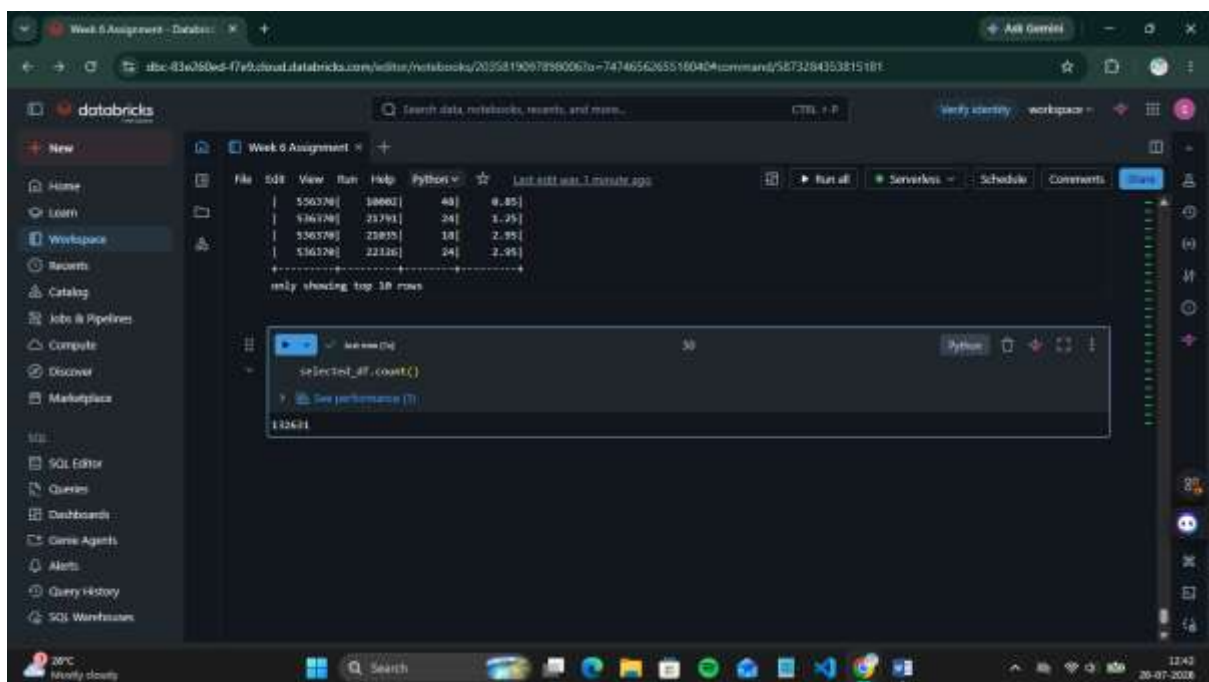
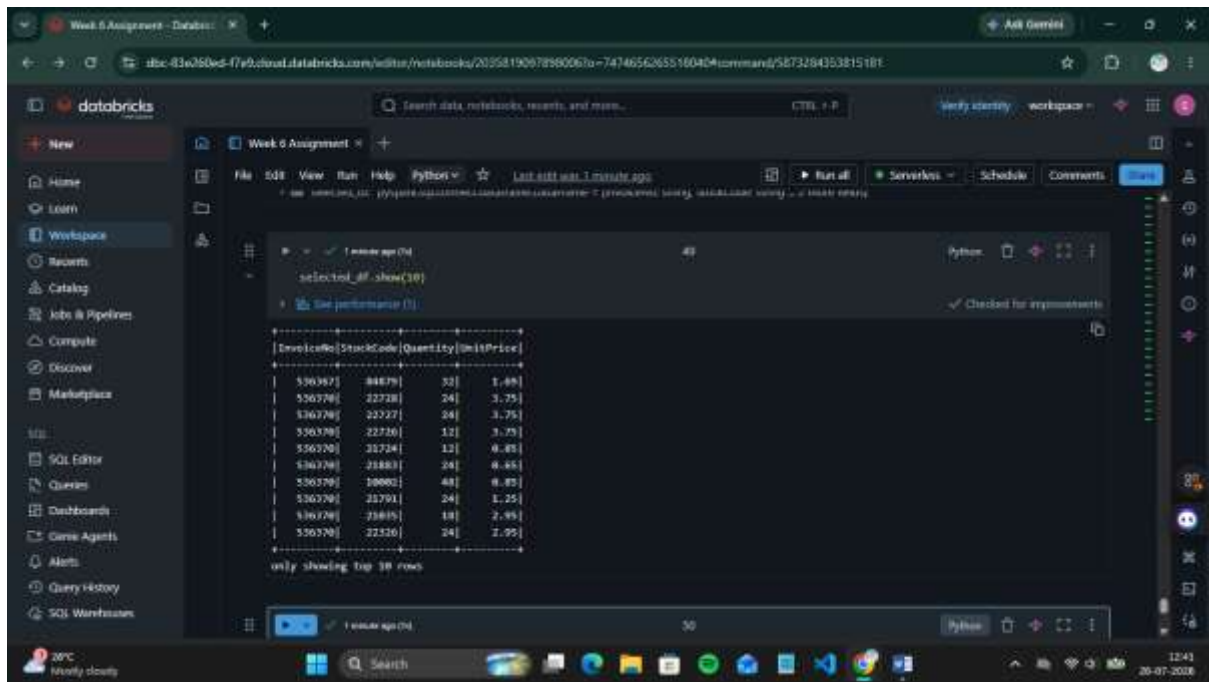
The screenshot displays the Databricks workspace interface. The notebook, titled 'Week 6 Assignment', contains the following Python code:

```
from pyspark.sql.functions import col

# Transformation 1
filtered_df = df.filter(col("Quantity") > 10)

# Transformation 2
selected_df = filtered_df.select(
    "InvoiceNo",
    "StockCode",
    "Quantity",
    "UnitPrice"
)
```

The output of the notebook shows the schema and data for the 'filtered_df' and 'selected_df' DataFrames. The 'filtered_df' schema is: InvoiceNo: string, StockCode: string, Quantity: integer, UnitPrice: double. The 'selected_df' schema is: InvoiceNo: string, StockCode: string, Quantity: integer, UnitPrice: double. The output also includes a 'show(10)' command and a 'See performance' link.



Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Answer

The following Spark code reads a Parquet file, removes rows where the `user_id` is null, and saves the filtered data as a CSV file.

```
from pyspark.sql.functions import col
```

```
spark.read.parquet("path/to/input") \
    .filter(col("user_id").isNotNull()) \
    .write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("path/to/output")
```

Explanation

- **spark.read.parquet("path/to/input")**: Reads the Parquet file from the specified location.
- **filter(col("user_id").isNotNull())**: Removes all rows where the user_id column contains null values.
- **write**: Writes the processed DataFrame to a file.
- **mode("overwrite")**: Replaces the existing output if the folder already exists.
- **option("header", "true")**: Includes column names as the first row in the CSV file.
- **csv("path/to/output")**: Saves the filtered data in CSV format at the specified output path.

Conclusion

This code demonstrates a common Spark data processing workflow: reading data from a **Parquet** file, filtering invalid records, and saving the cleaned data as a **CSV** file for further analysis or sharing.

Code:

```
parquet_df = spark.read.parquet(

    "/Volumes/workspace/default/ecommerce_volume/parquet_data"

)

display(parquet_df)


from pyspark.sql.functions import col

filtered_df = parquet_df.filter(

    col("CustomerID").isNotNull()

)

display(filtered_df)


filtered_df.write \

    .    mode("overwrite") \
```

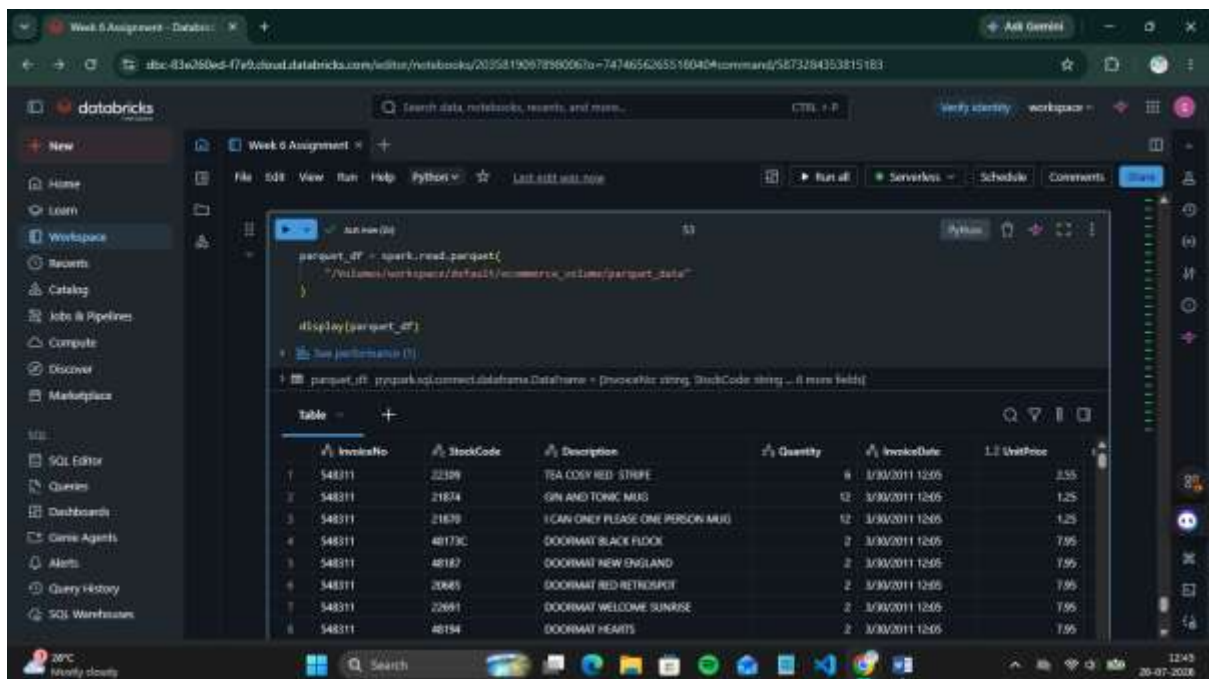
```
.option("header", "true") \
```

```
.csv("/Volumes/workspace/default/ecommerce_volume/output_csv")
```

```
print("Total Records after removing NULL CustomerID:")
```

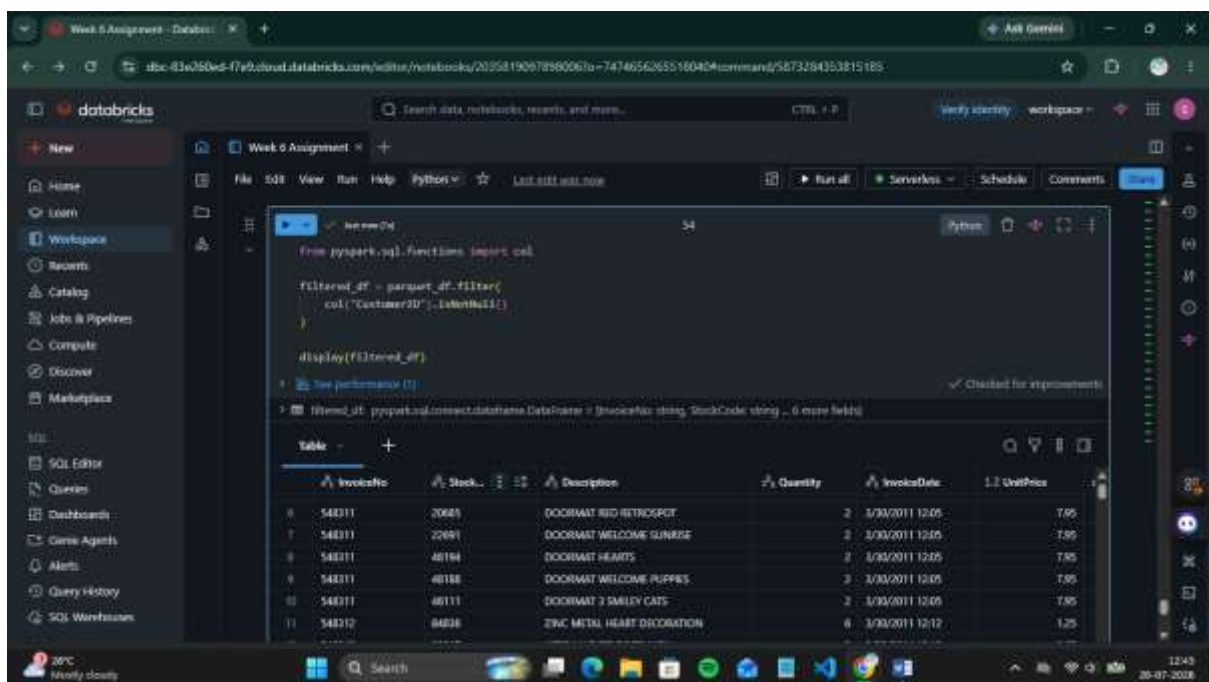
```
print(filtered_df.count())
```

Output:



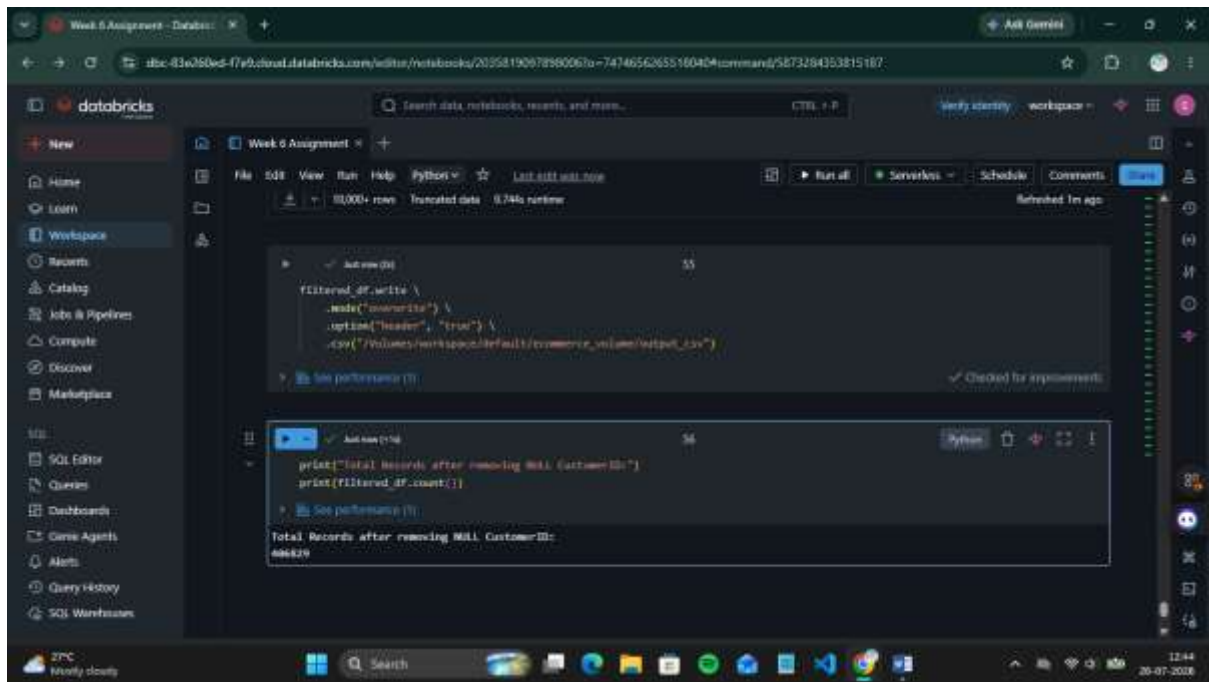
The screenshot shows the Databricks interface with a table view of the data. The table has 8 rows and 7 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice. The data is as follows:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
548311	22399	TSA COXY RED STRIPE	6	3/30/2011 12:05	2.55
548311	21874	GIN AND TONIC MUG	12	3/30/2011 12:05	1.25
548311	21870	I CAN ONLY PLEASE ONE PERSON MUG	12	3/30/2011 12:05	1.25
548311	48173C	DOORMAT BLACK FLOCK	2	3/30/2011 12:05	7.95
548311	48182	DOORMAT NEW ENGLAND	2	3/30/2011 12:05	7.95
548311	20685	DOORMAT RED RETROSPOT	2	3/30/2011 12:05	7.95
548311	22691	DOORMAT WELCOME SUNRISE	2	3/30/2011 12:05	7.95
548311	48194	DOORMAT HEARTS	2	3/30/2011 12:05	7.95



The screenshot shows the Databricks interface with a table view of the filtered data. The table has 7 rows and 7 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice. The data is as follows:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
548311	20685	DOORMAT RED RETROSPOT	2	3/30/2011 12:05	7.95
548311	22691	DOORMAT WELCOME SUNRISE	2	3/30/2011 12:05	7.95
548311	48194	DOORMAT HEARTS	2	3/30/2011 12:05	7.95
548311	48188	DOORMAT WELCOME PUPPES	2	3/30/2011 12:05	7.95
548311	48111	DOORMAT 2 DAILEY CATS	2	3/30/2011 12:05	7.95
548312	84838	ZINC METAL HEART DECORATION	6	3/30/2011 12:12	1.25



Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer

In Apache Spark, **Client Mode** and **Cluster Mode** refer to where the **Driver program** runs when a Spark application is submitted.

Client Mode

- The **Driver** runs on the **client machine** from which the application is submitted.
- The client machine must remain connected until the application finishes.
- It is mainly used for **development, testing, and debugging**.

Cluster Mode

- The **Driver** runs inside the **cluster** on one of the worker nodes.
- The client can disconnect after submitting the application.
- It is mainly used for **production environments** because it is more reliable and fault-tolerant.

Difference Between Client Mode and Cluster Mode

Client Mode	Cluster Mode
Driver runs on the client machine	Driver runs inside the cluster
Client must stay connected	Client can disconnect after job submission
Best for development and testing	Best for production environments

Client Mode

Less fault tolerant

Cluster Mode

More fault tolerant

Conclusion

The main difference is the location of the **Driver**. **Client Mode** is suitable for development and debugging, while **Cluster Mode** is preferred for production because it provides better reliability and resource management.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Answer

The following Spark query filters the DataFrame to return rows where the **region** is "**North**" **OR** the **priority** is "**High**".

```
from pyspark.sql.functions import col
```

```
df.filter(  
    (col("region") == "North") |  
    (col("priority") == "High")  
)
```

Explanation

- **filter()**: Used to filter rows based on a condition.
- **col("region") == "North"**: Selects rows where the region is **North**.
- **col("priority") == "High"**: Selects rows where the priority is **High**.
- **| (OR operator)**: Returns rows that satisfy **either** of the two conditions.

Example

Input Data:

Order ID	Region	Priority
----------	--------	----------

101	North	Low
-----	-------	-----

102	South	High
-----	-------	------

103	North	High
-----	-------	------

104	East	Medium
-----	------	--------

Output:

Order ID Region Priority

101	North	Low
102	South	High
103	North	High

Conclusion

This query uses the **OR () operator** to filter data based on multiple conditions. It returns all rows where **either the region is "North" or the priority is "High"**.

Code:

```
from pyspark.sql.functions import col

filtered_df = df.filter(

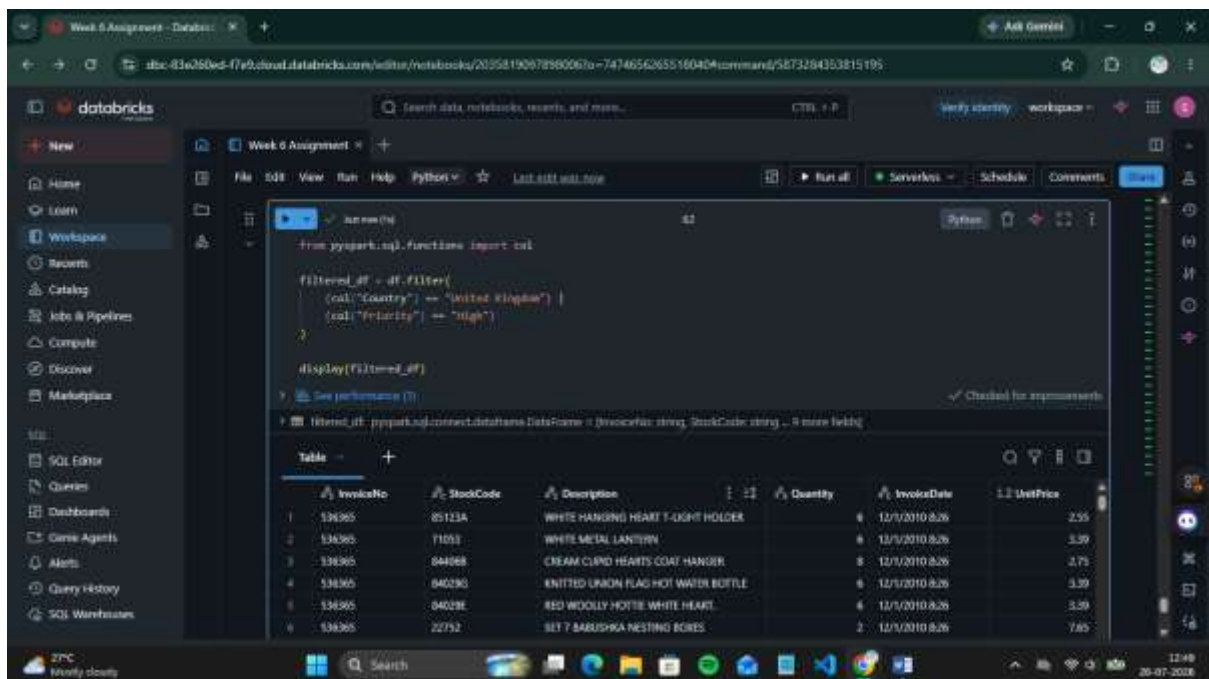
    (col("Country") == "United Kingdom") |

    (col("Priority") == "High")

)

display(filtered_df)
```

Output:



The screenshot shows a Databricks workspace interface. The top navigation bar includes 'New', 'Home', 'Learn', 'Workspace', 'Recent', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', and 'Marketplace'. The left sidebar contains 'SQL Editor', 'Queries', 'Dashboards', 'Game Agents', 'Alerts', 'Query History', and 'SQL Warehouse'. The main area displays a PySpark code editor with the following code:

```
from pyspark.sql.functions import col

filtered_df = df.filter(
    (col("Country") == "United Kingdom") |
    (col("Priority") == "High")
)

display(filtered_df)
```

Below the code editor, a table is displayed with the following columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, and UnitPrice. The table contains 6 rows of data:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
136365	851234	WHITE HANGING HEART T-LIGHT HOLDER	4	12/1/2010 8:26	2.55
136365	71051	WHITE METAL LANTERN	4	12/1/2010 8:26	3.59
136365	844068	CREAM CLOUD HEARTS COAT HANGER	8	12/1/2010 8:26	2.75
136365	840290	KNITTED UNION FLAG HOT WATER BOTTLE	4	12/1/2010 8:26	3.59
136365	840291	RED WOOLLY HOTTIE WHITE HEART	4	12/1/2010 8:26	3.59
136365	22752	SILY T-BARUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer

When working with very large datasets in Apache Spark, it is much safer to use `.show(5)` instead of `.collect()` because `.show(5)` displays only a few rows, while `.collect()` retrieves the entire dataset to the Driver.

`.show(5)`

- Displays only the **first 5 rows** of the DataFrame.
- Transfers only a small amount of data to the Driver.
- Uses less memory and executes quickly.
- Ideal for previewing or exploring large datasets.

`.collect()`

- Retrieves **all rows** from the DataFrame to the Driver.
- Can consume a large amount of memory.
- On multi-terabyte datasets, it may cause an **OutOfMemoryError** or even crash the application.
- Suitable only for small datasets.

Difference

<code>.show(5)</code>	<code>.collect()</code>
Displays only the first 5 rows	Retrieves the entire dataset
Uses very little memory	Uses a large amount of memory
Safe for large datasets	Suitable only for small datasets
Mainly used for data preview	Used when all data is required in the Driver

Conclusion

For exploring large datasets, `.show(5)` is the safer and more efficient choice because it displays only a small sample of the data. In contrast, `.collect()` loads the entire dataset into the Driver's memory, which can lead to performance issues or application failure when working with very large datasets.

Code:

```
# Display only the first 5 rows

df.show(5, truncate=False)
```

Collect all records to the Driver (Not recommended for large datasets)

```
all_rows = df.collect()
```

```
print("Total Rows Collected:", len(all_rows))
```

Output:

